AI Customer Support Copilot

System Design and Implementation Plan With Issued Ticket Support and Real-Time Live Chat Assistant

1. Project Overview

The AI Customer Support Copilot is a human-in-the-loop AI system designed to assist customer support teams. The system helps agents respond to customer issues faster, more accurately, and more consistently by analyzing customer messages, retrieving reliable knowledge, generating response drafts, recommending escalation, and learning from agent feedback.

The system supports two major workflows:

1. **Issued Ticket Copilot Mode**
   Used for support tickets submitted through email, form, CSV upload, or support platforms.

2. **Real-Time Live Chat Assistant Mode**
   Used during active live chat conversations between customers and support agents.

The AI Copilot does not directly send customer-facing replies in the MVP. Instead, it generates suggestions, explanations, sources, confidence scores, and escalation warnings for the support agent. The human agent reviews, edits, accepts, or rejects the AI recommendation before responding to the customer.

---

2. Problem Statement

Support teams often face repeated questions, scattered documentation, slow response times, inconsistent answers, missed urgent issues, and difficulty reviewing long customer conversations. Traditional support workflows require agents to manually search policies, FAQs, product documents, and previous tickets while also maintaining a polite and accurate response.

This project solves that problem by building an AI assistant that helps support agents during both asynchronous ticket handling and real-time live chat support.

---

3. Product Vision

The final system should work like an intelligent assistant inside a support platform. When a customer issue arrives, the copilot understands the issue, retrieves relevant knowledge, generates a safe and source-grounded draft, recommends escalation if needed, and learns from agent feedback.

The system should:

- Understand customer intent, category, priority, sentiment, language, and key entities.
- Retrieve trustworthy information from FAQs, policies, product documentation, known issues, and past resolved tickets.
- Generate editable response drafts with citations and confidence scores.
- Assist agents during live chat in real time.
- Detect risky cases such as payment disputes, security issues, legal concerns, VIP customers, angry customers, and outages.
- Keep the human support agent in control.
- Collect feedback from agent actions and use it for improvement.
- Provide analytics for support managers and product teams.

---

4. System Operating Modes

4.1 Issued Ticket Copilot Mode

Issued Ticket Mode handles customer requests submitted through channels such as:

- Email
- Support form
- CSV upload
- Support portal
- Zendesk, Freshdesk, Intercom, or similar tools

Workflow

1. Customer submits a ticket.
2. Ticket ingestion service receives the ticket.
3. Preprocessing layer cleans the message and masks sensitive data.
4. Ticket intelligence layer predicts intent, category, priority, sentiment, language, and entities.
5. RAG layer retrieves relevant policies, FAQs, documents, known issues, and similar past tickets.
6. LLM response layer generates an editable draft.

7. Confidence scoring and escalation engine decide whether the issue is safe to answer or should be escalated.
8. Agent reviews the draft, sources, confidence score, missing information, and escalation reason.
9. Agent accepts, edits, regenerates, rejects, or escalates.
10. Feedback is stored for analytics and future improvement.

Example

Customer ticket:

> I was charged twice yesterday and I still have not received my refund. This is really frustrating.

AI output:

```
{
  "category": "Billing",
  "intent": "refund_request",
  "priority": "High",
  "sentiment": "Frustrated",
  "entities": {
    "issue": "duplicate charge",
    "time": "yesterday",
    "requested_action": "refund"
  },
  "missing_info": ["transaction_id"],
  "escalation_required": false
}
```

4.2 Real-Time Live Chat Assistant Mode

Live Chat Assistant Mode supports agents during active customer conversations. Instead of waiting for a full ticket to be submitted, the system observes the live conversation and continuously assists the agent.

Workflow

1. Customer sends a message in live chat.
2. Live chat stream listener receives the new message.
3. Real-time context builder updates the conversation history.
4. Ticket intelligence layer analyzes the latest message and full conversation context.
5. RAG layer retrieves related FAQs, policies, product documents, known issues, and past resolved cases.
6. LLM response layer generates suggested replies.

7. The agent sees suggested responses, source snippets, sentiment, urgency, missing information, and escalation warnings.
8. Agent accepts, edits, ignores, or escalates.
9. Feedback is stored for improvement.

Important Rule

The AI should not automatically send messages to the customer in the MVP. It should only suggest replies to the support agent.

Example

Customer live chat message:

> My account was charged twice. I need this fixed now.

AI assistant panel shows:

```
{
  "suggested_reply": "I’m sorry for the trouble. I understand how frustrating
a duplicate charge can be. Could you please share the transaction ID or
payment receipt so we can verify the payment and help process the refund if
the duplicate charge is confirmed?",
  "intent": "refund_request",
  "category": "Billing",
  "priority": "High",
  "sentiment": "Frustrated",
  "retrieved_sources": [
    "Refund Policy - Duplicate Charges",
    "Past Ticket - Duplicate billing after payment retry"
  ],
  "missing_info": ["transaction_id", "payment_receipt"],
  "escalation_required": false
}
```

---

5. Updated High-Level Architecture

6. Main System Modules

6.1 Ticket Ingestion Service

This module receives customer tickets from email, form submission, CSV upload, or external support platforms.

Responsibilities

- Accept new tickets.
- Store ticket metadata.

- Normalize ticket format.
- Support manual entry and CSV upload for MVP.
- Optionally integrate with Zendesk, Freshdesk, Intercom, or Gmail in the advanced version.

Example Ticket Payload

```
{
  "ticket_id": "TCK-001",
  "customer_name": "John Smith",
  "customer_email": "john@example.com",
  "message": "I was charged twice and I want a refund.",
  "channel": "email",
  "customer_plan": "premium",
  "created_at": "2026-05-28T10:00:00Z"
}
```

6.2 Live Chat Stream Listener

This is the new feature module for real-time support.

Responsibilities

- Listen to live chat messages.
- Capture customer and agent messages.
- Update the conversation state after every new message.
- Trigger real-time AI analysis.
- Send AI suggestions to the agent panel.
- Avoid sending AI messages directly to the customer.

Example Live Chat Event

```
{
  "conversation_id": "CHAT-101",
  "sender": "customer",
  "message": "I cannot log in and I need access urgently.",
  "timestamp": "2026-05-28T10:05:00Z",
  "customer_plan": "enterprise"
}
```

6.3 Real-Time Context Builder

Live chat messages are short and depend heavily on previous messages. This module maintains the conversation context.

Responsibilities

- Store recent conversation history.
- Summarize long conversations.
- Track unresolved questions.
- Track already requested information.
- Avoid repeating the same suggestion.
- Update the issue summary after each customer message.

Example Context Summary

```
{
  "conversation_id": "CHAT-101",
  "current_issue": "Customer cannot log in urgently",
  "known_details": {
    "customer_plan": "enterprise",
    "issue_type": "login_issue"
  },
  "missing_details": ["error_message", "account_email"],
  "latest_sentiment": "frustrated"
}
```

6.4 Preprocessing and Privacy Layer

This layer prepares customer text for AI processing and protects sensitive information.

Tasks

- Clean HTML and email formatting.
- Remove duplicate signatures and boilerplate.
- Detect language.
- Split conversations into turns.
- Mask PII such as email, phone number, card number, address, API key, and customer ID.
- Normalize error codes, product names, and timestamps.

Masked Data Examples

Sensitive Data	Masked Format
Email address	[EMAIL]
Phone number	[PHONE]
Credit card	[CARD]
Address	[ADDRESS]
API key	[SECRET]

Sensitive Data	Masked Format
Customer ID	[CUSTOMER_ID]

6.5 Ticket Intelligence Layer

This layer understands the customer issue before generating an answer.

Components

Component	Output Example
Intent classifier	refund_request, login_issue, bug_report
Category classifier	Billing, Account, Technical, Security
Priority classifier	Low, Medium, High, Urgent
Sentiment detector	Angry, Frustrated, Confused, Neutral
Entity extractor	order_id, amount, product, error_code
Language detector	en, my, ja, th

Priority Rules

Signal	Likely Priority
Production is down	Urgent
Security or account takeover	Urgent
Duplicate charge or refund issue	High
Cannot log in	High
General how-to question	Low
Minor UI bug	Low or Medium
Enterprise or VIP customer issue	High or Urgent

6.6 Knowledge Base and RAG Retrieval Layer

This layer retrieves trustworthy information so the AI does not guess.

Knowledge Sources

- Product documentation
- FAQs
- Refund policy
- Privacy policy
- SLA rules
- Troubleshooting guides

- Internal playbooks
- Past resolved tickets
- Known issues and outage reports

Retrieval Strategy

Use hybrid retrieval:

1. **Vector search** for semantic similarity.
2. **Keyword/BM25 search** for exact matches such as error codes, policy names, and product names.
3. **Metadata filtering** by product, category, customer plan, date, language, and access level.
4. **Reranking** to select the best evidence.
5. **Freshness filtering** to prefer updated documents and active known issues.

Output


```
{
  "retrieved_context": [
    {
      "source": "Refund Policy",
      "section": "Duplicate Charges",
      "text": "Duplicate charges are eligible for refund after payment
verification."
    },
    {
      "source": "Past Ticket TCK-384",
      "summary": "Customer had duplicate billing after payment retry. Agent
verified transaction ID."
    }
  ]
}
```

6.7 LLM Response Generation Layer

This layer creates the draft response shown to the support agent.

Responsibilities

- Generate customer-facing reply draft.
- Generate private agent notes.
- Include citations.
- Identify missing information.
- Recommend escalation.

- Support tone control.
- Support multilingual replies.
- Avoid unsupported claims.

Prompt Structure

```
System:
You are a customer support copilot. Generate replies only from the provided
evidence.

Context:
Retrieved policies, FAQs, product documents, known issues, and similar past
tickets.

Ticket or Chat Context:
Customer message, conversation history, customer metadata, detected intent,
sentiment, priority, and entities.

Rules:
- Do not promise refunds unless the policy allows it.
- Do not provide legal, security, or privacy decisions without escalation.
- Ask for missing information when required.
- Do not send the reply directly to the customer.
- Include citations and confidence score.

Output JSON:
reply_draft, agent_notes, citations, confidence, missing_info, escalation
```

6.8 Safety, Guardrails, and Escalation Layer

This layer decides whether the AI suggestion is safe and complete.

Checks

- Retrieval confidence
- Policy compliance
- PII leakage
- Hallucination risk
- Sentiment risk
- Legal, privacy, security, or payment risk
- VIP customer handling
- Repeated similar tickets indicating possible outage
- Conflicting classifier results

Escalation Triggers

Trigger	Example
Low retrieval confidence	No relevant policy found
High-risk topic	Legal, security, privacy
Strong negative sentiment	Customer threatens to cancel
Possible outage	Many similar tickets in short time
VIP customer	Enterprise account issue
Financial risk	Large refund or billing dispute
Model uncertainty	Conflicting classification results

6.9 Agent Review Dashboard

This UI is used for issued tickets.

Features

- Ticket details
- Customer metadata
- AI analysis panel
- Retrieved evidence panel
- Similar past ticket panel
- Draft reply editor
- Confidence score
- Missing information
- Escalation reason
- Accept, edit, reject, regenerate, or escalate buttons
- Feedback rating

6.10 Live Chat Suggestion Panel

This UI is used during real-time support conversations.

Features

- Ongoing customer-agent conversation
- Real-time intent and sentiment
- Suggested reply drafts
- Quick reply options
- Relevant policy snippets

- Similar past cases
- Missing information suggestions
- Escalation warnings
- Customer mood indicator
- “Ask for more details” suggestion
- “Transfer to specialist” recommendation
- Accept, edit, ignore, or regenerate buttons

Example Agent View

```
Customer: I was charged twice. I need this fixed now.

AI Suggestions:
1. Apologize and acknowledge frustration.
2. Ask for transaction ID or receipt.
3. Mention that duplicate charges can be investigated.
4. Do not promise refund until verification.

Suggested Reply:
I’m sorry for the trouble. I understand how frustrating a duplicate charge
can be. Could you please share the transaction ID or payment receipt so we
can verify the payment and help process the refund if the duplicate charge is
confirmed?

Confidence: 0.84
Escalation: Not required
Missing Info: transaction_id, payment_receipt
Sources: Refund Policy - Duplicate Charges
```

6.11 Feedback and Learning Layer

The system stores how agents interact with AI outputs.

Feedback Events

Event	Meaning
Agent accepted draft	AI response was useful
Agent edited draft	AI response was partially useful
Agent rejected draft	AI response failed
Agent escalated	Escalation model can learn
Customer replied positively	Possible success signal
Ticket reopened	Possible failure signal
Live chat suggestion ignored	Suggestion may be irrelevant

Event	Meaning
Live chat suggestion used	Suggestion helped agent respond faster

Uses

- Improve prompt templates.
- Build fine-tuning datasets.
- Improve classifier accuracy.
- Improve retrieval quality.
- Track AI usefulness.
- Measure agent productivity.

---

7. Data Storage Design

Use relational storage for structured data and vector storage for semantic retrieval.

Recommended Storage

Storage	Purpose
PostgreSQL	Tickets, customers, feedback, predictions, audit logs
pgvector, Chroma, Qdrant, or FAISS	Embeddings and vector retrieval
Object storage	Uploaded files, attachments, raw documents
Analytics database	Aggregated dashboard metrics

Example Tables

```
customers(
  customer_id,
  name,
  email_hash,
  plan,
  created_at
)

tickets(
  ticket_id,
  customer_id,
  channel,
  subject,
  message,
  status,
  created_at
)
```

```
live_chat_conversations(
  conversation_id,
  customer_id,
  agent_id,
  status,
  started_at,
  ended_at
)

live_chat_messages(
  message_id,
  conversation_id,
  sender,
  message_text,
  timestamp
)

ticket_predictions(
  ticket_id,
  intent,
  category,
  priority,
  sentiment,
  language,
  confidence
)

chat_predictions(
  prediction_id,
  conversation_id,
  message_id,
  intent,
  category,
  priority,
  sentiment,
  confidence,
  created_at
)

knowledge_docs(
  doc_id,
  title,
  source_type,
  version,
  created_at,
  updated_at
)
```

```
knowledge_chunks(
  chunk_id,
  doc_id,
  chunk_text,
  metadata,
  embedding_id
)

similar_ticket_links(
  ticket_id,
  similar_ticket_id,
  similarity_score
)

ai_drafts(
  draft_id,
  ticket_id,
  reply_text,
  confidence,
  citations,
  created_at
)

live_chat_suggestions(
  suggestion_id,
  conversation_id,
  message_id,
  suggested_reply,
  confidence,
  citations,
  created_at
)

agent_feedback(
  feedback_id,
  draft_id,
  suggestion_id,
  action,
  rating,
  edited_text,
  created_at
)

escalations(
  escalation_id,
  ticket_id,
  conversation_id,
  reason,
  target_team,
```

```
  status
)

audit_logs(
  log_id,
  actor,
  action,
  object_type,
  object_id,
  timestamp
)
```

---

8. API Design

Ticket APIs

Endpoint	Method	Purpose
/tickets	POST	Create or upload ticket
/tickets/{id}	GET	Get ticket details
/tickets/{id}/analyze	POST	Analyze ticket
/tickets/{id}/retrieve	POST	Retrieve related knowledge
/tickets/{id}/draft	POST	Generate response draft
/tickets/{id}/escalate	POST	Escalate ticket

Live Chat APIs

Endpoint	Method	Purpose
/chat/conversations	POST	Start live chat conversation
/chat/conversations/{id}	GET	Get conversation
/chat/conversations/{id}/messages	POST	Add chat message
/chat/conversations/{id}/analyze	POST	Analyze conversation context
/chat/conversations/{id}/suggest	POST	Generate real-time reply suggestion
/chat/suggestions/{id}/feedback	POST	Submit feedback on suggestion

Knowledge APIs

Endpoint	Method	Purpose
/knowledge/upload	POST	Upload policies, FAQs, docs

Endpoint	Method	Purpose
/knowledge/reindex	POST	Rebuild embeddings
/knowledge/search	POST	Search knowledge base

Feedback and Analytics APIs

Endpoint	Method	Purpose
/drafts/{id}/feedback	POST	Submit ticket draft feedback
/analytics/overview	GET	Get dashboard overview
/analytics/live-chat	GET	Get live chat performance
/analytics/evaluation	GET	Get AI evaluation metrics

---

9. Model Strategy

Use multiple AI models instead of relying only on one LLM prompt.

Task	MVP Approach	Advanced Approach
Intent classification	LLM prompt or scikit-learn	Fine-tuned BERT, DistilBERT, or Gemma
Category classification	LLM prompt or traditional ML	Fine-tuned transformer
Sentiment detection	Pretrained sentiment model	Fine-tuned support sentiment model
Priority prediction	Rules	Rules + ML + LLM verification
Entity extraction	Regex + LLM JSON	NER + validation
Knowledge retrieval	Vector search	Hybrid search + reranking
Response generation	RAG prompt	RAG + guardrails + self-check
Live chat suggestions	Latest message + context	Real-time context summarization + retrieval
Evaluation	Manual testing	RAGAS, DeepEval, LLM-as-judge, human review
Learning	Feedback logs	Fine-tuning from accepted and edited replies

---

10. RAG Pipeline Design

Stages

1. Upload documents, FAQs, policies, resolved tickets, and known issues.
2. Clean and normalize documents.
3. Split documents into meaningful chunks.
4. Add metadata such as source type, product, date, language, and access level.
5. Generate embeddings.
6. Store embeddings in vector database.
7. For each ticket or live chat message, build a query using issue summary, intent, entities, and error codes.
8. Retrieve relevant chunks using hybrid search.
9. Rerank results.
10. Generate grounded draft using selected evidence.
11. Show citations to the agent.

---

11. Evaluation Plan

The project should be evaluated like a real AI system.

Area	Metrics
Classification	Accuracy, precision, recall, F1-score
Priority detection	Urgent recall, false negative rate
Retrieval	Hit rate, context precision, context recall, MRR
Generation	Faithfulness, helpfulness, tone quality, policy compliance
Live chat	Suggestion acceptance rate, average response time reduction
Human review	Agent acceptance rate, edit distance, thumbs up/down
Safety	Hallucination rate, PII leakage rate, escalation accuracy
Business impact	Average response time, first contact resolution, reopen rate

Live Chat-Specific Evaluation

Metric	Meaning
Suggestion latency	Time to generate suggestion after customer

Metric	Meaning
	message
Suggestion acceptance rate	Percentage of suggestions used by agents
Agent edit distance	How much agents modify AI replies
Conversation resolution time	Time taken to solve live chat issue
Escalation accuracy	Whether risky chats are correctly flagged

---

12. Security and Enterprise Readiness

To make the system enterprise-grade, include:

- Authentication and authorization.
- Role-based access control.
- PII masking before LLM processing.
- Audit logs for every AI draft, suggestion, source, and agent action.
- Human approval before sending replies.
- Encryption for stored data.
- Rate limiting.
- Monitoring for latency, cost, errors, and model failures.
- Model version tracking.
- Prompt version tracking.
- Knowledge document versioning.
- Secure API keys and secrets management.
- Escalation for legal, security, privacy, and high-value billing issues.

---

13. Recommended Tech Stack

MVP Stack

Layer	Tools
UI	Streamlit or Gradio
Backend	FastAPI
Database	SQLite or PostgreSQL
Vector DB	Chroma or FAISS
Embeddings	sentence-transformers
LLM	Gemini, GPT, Llama, or Gemma
Classifier	scikit-learn or DistilBERT
Evaluation	Custom metrics, RAGAS, DeepEval

More Enterprise-Style Stack

Layer	Tools
Frontend	React, Next.js, Tailwind CSS
Backend	FastAPI or Node.js
Database	PostgreSQL
Vector DB	pgvector, Qdrant, or Chroma
Queue	Celery, Redis Queue, or BullMQ
Deployment	Docker, AWS, GCP, Azure, Render, or Railway
Analytics	Metabase, Superset, or custom dashboard
Monitoring	Prometheus, Grafana, OpenTelemetry

---

14. MVP Scope

The MVP should be realistic and demo-friendly.

MVP Features

Feature	Description
Ticket upload/form	Enter or upload support tickets
Live chat demo panel	Simulate customer-agent chat
Ticket analysis	Intent, category, priority, sentiment, entities
Knowledge upload	Upload FAQs, policies, docs
RAG retrieval	Retrieve relevant sources
Reply draft	Generate editable support response
Live chat suggestion	Generate real-time suggested replies
Citations	Show supporting sources
Confidence score	Show AI certainty
Escalation flag	Detect risky or uncertain cases
Feedback buttons	Accept, edit, reject, rate

---

15. Advanced Features

Feature	Why It Is Impressive
Real-time live chat assistant	Shows dynamic AI support, not just static ticket processing
Similar past ticket retrieval	Reuses previous successful resolutions
Hybrid search and reranking	Improves retrieval quality

Feature	Why It Is Impressive
Fine-tuned classifier	Shows custom ML, not just prompting
Multilingual replies	Supports international users
Outage cluster detection	Detects many users reporting same issue
Agent tone personalization	Learns preferred support style
Product feedback mining	Converts support issues into product insights
A/B evaluation dashboard	Compares AI-assisted vs traditional support
Hallucination checker	Improves trust and safety

---

16. Four-Member Team Role Breakdown

Member 1: Ticket Intelligence Engineer

Owns

- Intent detection
- Category classification
- Priority prediction
- Sentiment detection
- Language detection
- Entity extraction

Deliverables

- Ticket label taxonomy
- Labeled dataset
- Classification model
- Evaluation report
- Ticket and chat analysis API

Member 2: RAG and Knowledge Engineer

Owns

- Document ingestion
- FAQ ingestion
- Policy ingestion
- Past ticket ingestion
- Chunking strategy

- Embeddings
- Vector database
- Hybrid search
- Similar ticket retrieval
- Citations

Deliverables

- Knowledge base pipeline
- Vector database
- Retrieval API
- Similar case search
- Source citation system
- Retrieval evaluation

Member 3: LLM Response and Live Chat Engineer

Owns

- Prompt engineering
- Ticket draft generation
- Live chat response suggestions
- Tone control
- Confidence scoring
- Escalation decision
- Hallucination reduction
- JSON output formatting

Deliverables

- Reply generation prompts
- Live chat suggestion engine
- Tone variations
- Escalation rules
- Confidence scoring system
- Draft evaluation

Member 4: Evaluation, Feedback, and Analytics Engineer

Owns

- Evaluation framework

- Human feedback loop
- Error analysis
- Model comparison
- Fine-tuning dataset preparation
- Analytics dashboard
- A/B testing logic

Deliverables

- Evaluation dashboard
- Test dataset
- Benchmark report
- Feedback loop
- Fine-tuning dataset
- Final performance comparison

---

17. Implementation Roadmap

Phase 1: Basic Ticket Processing

Deliverables

- Ticket form
- CSV upload
- Basic database schema
- Preprocessing and PII masking

Phase 2: Ticket Intelligence

Deliverables

- Intent classifier
- Category classifier
- Priority classifier
- Sentiment detector
- Entity extractor

Phase 3: Knowledge Base and Retrieval

Deliverables

- Document upload
- Chunking

- Embeddings
- Vector database
- Retrieval API
- Source citations

Phase 4: RAG Reply Generation

Deliverables

- Draft reply generator
- Agent notes
- Missing information detection
- Confidence score
- Escalation recommendation

Phase 5: Live Chat Assistant

Deliverables

- Live chat simulation UI
- Conversation stream listener
- Real-time context builder
- Real-time intent and sentiment updates
- Suggested reply panel
- Live chat feedback collection

Phase 6: Human Review UI

Deliverables

- Agent ticket dashboard
- Draft editor
- Accept, edit, reject, regenerate, escalate buttons
- Evidence viewer
- Similar past ticket viewer

Phase 7: Feedback and Analytics

Deliverables

- Feedback logging
- AI acceptance rate
- Top issue dashboard
- Live chat response time analytics

- Escalation analytics
- Agent productivity metrics

Phase 8: Advanced Modeling

Deliverables

- Fine-tuned classifier
- Hybrid search and reranking
- Multilingual support
- Outage detection
- Fine-tuning dataset from feedback

---

18. Demo Scenario

A strong final demo should show both modes.

Demo Part 1: Issued Ticket Mode

1. Upload refund policy, FAQ, product docs, and past resolved tickets.
2. Create a ticket: “I was charged twice and I want a refund.”
3. System classifies it as Billing, refund request, High priority, Frustrated sentiment.
4. System retrieves refund policy and similar past ticket.
5. System generates a reply asking for transaction ID.
6. Agent sees citations, confidence, and missing info.
7. Agent edits one sentence and accepts.
8. Dashboard updates acceptance rate and category count.

Demo Part 2: Real-Time Live Chat Mode

1. Start a simulated live chat.
2. Customer says: “I can’t log in and I need access urgently.”
3. System detects Account/Login issue, High priority, Frustrated sentiment.
4. System retrieves login troubleshooting guide and account recovery policy.
5. AI suggests a real-time reply asking for email and error message.
6. Customer sends more details.
7. System updates context and generates next reply.
8. Agent accepts the suggestion.
9. Analytics dashboard updates live chat suggestion acceptance and response time.

---

19. Final Project Pitch

Our project is a Dual-Mode AI Customer Support Copilot for both issued tickets and real-time live chat support. It helps support agents understand customer issues, retrieve trusted knowledge, generate safe and editable response drafts, recommend escalation, and learn from agent feedback.

For issued tickets, the system analyzes the complete customer request and generates a source-grounded draft response. For live chat, the system assists the support agent in real time by continuously analyzing the conversation, retrieving relevant information, and suggesting replies during the conversation.

The system uses ticket classification, sentiment detection, entity extraction, RAG-based retrieval, LLM response generation, confidence scoring, safety checks, human review, feedback learning, and analytics. This makes the project more than a chatbot: it is an enterprise-style AI support assistant that improves speed, consistency, accuracy, and customer experience while keeping humans in control.